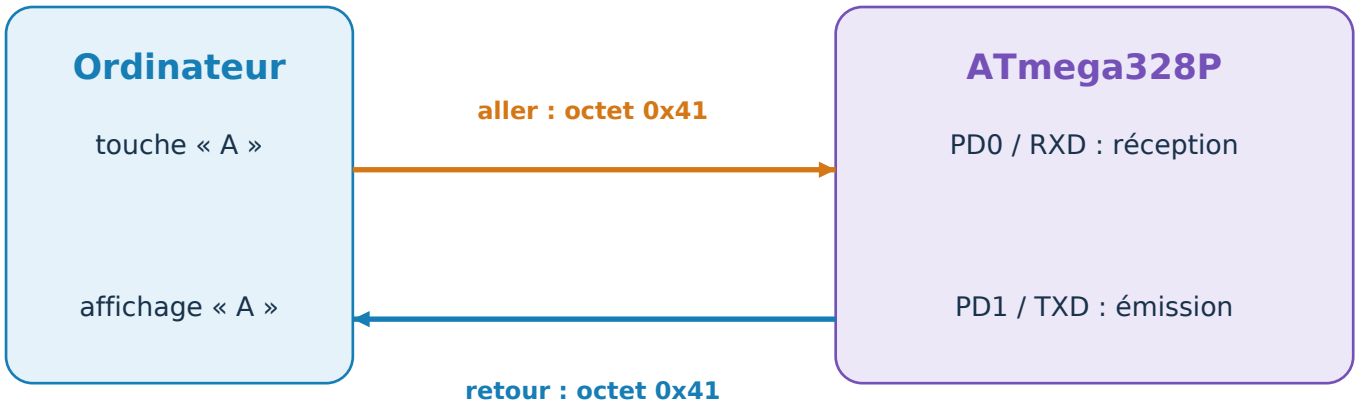
# Recevoir un octet, puis le renvoyer

Le PC envoie ; le MCU reçoit ; le MCU renvoie ; le PC affiche.

## Le nouvel objectif

Écrire une fonction **uart_rx** qui attend un caractère reçu et le retourne. Le programme utilise cette valeur et la transmet avec **uart_tx**, déjà étudiée. Ce retour du caractère vers son expéditeur s’appelle un **écho**.

## L’aller et le retour du même caractère




L’interface USB-UART entre le PC et le MCU est simplifiée dans ce schéma.

## Trois rôles à distinguer

**uart_rx :** attend et fournit un caractère.
**uart_tx :** transmet le caractère qu’on lui donne.
**Le programme :** relie la valeur reçue à la demande d’émission, puis recommence.

## La nouvelle contrainte est la réception

Cet énoncé ne demande ni une période de 2 secondes, ni une boucle principale vide. Une réception par **scrutation** suffit pour comprendre ce nouvel exercice ; un timer n’est pas nécessaire.

Sources : datasheet, §19.2 p.143-144 ; §19.6 p.150 ; §19.7 p.152

# Comprendre char uart_rx(void)

La valeur arrive par le retour de la fonction, pas par un argument.

```
char uart_rx(void);
```

| Élément | Signification |
|---|---|
| **char** | La fonction renvoie un caractère. |
| **uart_rx** | Nom de la fonction de réception. |
| **(void)** | La fonction ne reçoit aucun argument. |
| **;** | Déclaration : le corps de la fonction reste à écrire. |

### Recevoir : une sortie

```
char uart_rx(void);
```

Le périphérique apporte la donnée. Le résultat de l'appel contient le caractère reçu.

### Transmettre : une entrée

```
void uart_tx(char c);
```

L'appelant fournit le caractère c. La fonction ne renvoie aucune valeur.

### Le mot return

En C, **return expression;** termine la fonction et donne la valeur de l'expression à l'appelant. Pour l'écho, cette valeur est importante : appeler la fonction puis ignorer son résultat fait perdre le caractère à renvoyer.

### char ne signifie pas « chaîne »

Un appel fournit **un octet, y compris la valeur 0**. Une réception n'est pas une chaîne terminée par '\0'. Le signe de char dépend du compilateur ; aucun code négatif n'est prévu pour « rien reçu ».

# Le matériel reconstitue le caractère

Tu lis un octet complet ; tu n’as pas à lire chaque bit de PD0 en C.

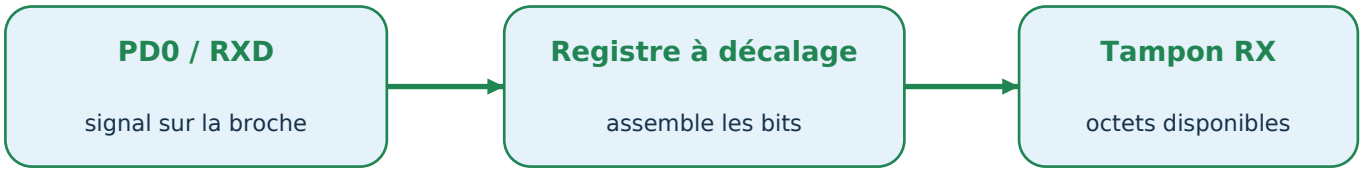
## Réception asynchrone

La ligne est à 1 au repos. Un start à 0 permet au récepteur de se synchroniser. Il échantillonne les bits suivants selon le débit configuré, puis vérifie le premier bit de stop. Les données sont placées dans le tampon de réception.

| start | d0 | d1 | d2 | d3 | d4 | d5 | d6 | d7 | stop |
|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 1 |

Exemple : « **A** » = **0x41**. Les bits de données arrivent avec **d0 en premier**. Le start et le stop servent à la trame ; ils ne deviennent pas des caractères supplémentaires.

## Le chemin interne de réception




## Quand un octet devient disponible

Détecter un start ne signifie pas encore qu’un caractère est prêt. Il faut que la réception de la trame aboutisse et que la donnée arrive dans le tampon. Le programme doit se fier à **l’indicateur de réception**, pas au niveau instantané de la broche.

En 8N1, une trame fait **10 bits**. À 115200 bit/s, elle dure environ **86,8 µs**. L’aller-retour ajoute une deuxième trame, plus les délais du programme et de l’interface PC.

Sources : datasheet, §19.4 p.147-148 ; §19.7.1 p.152 ; §19.8 p.155-156

# UCSR0B : il faut aussi recevoir

L’émetteur et le récepteur ont chacun leur autorisation.

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| RXCIE0 | TXCIE0 | UDRIE0 | RXEN0 | TXEN0 | UCSZ02 | RXB80 | TXB80 |

## RXEN0 : récepteur

Active la réception USART. **PD0 / RXD** devient une entrée du périphérique, indépendamment de DDD0.

## TXEN0 : émetteur

Active l’émission USART. **PD1 / TXD** devient une sortie du périphérique, indépendamment de DDD1.

## Une configuration TX seule ne reçoit pas

L’initialisation du premier exercice pouvait n’activer que l’émetteur. Ici, vérifie que l’initialisation prépare **les deux directions**. Écrire une nouvelle valeur de UCSR0B remplace les anciens réglages : il faut conserver ceux dont tu as besoin.

| Bit / champ | Ce qu’il ne faut pas confondre |
|---|---|
| **RXCIE0** | Autorisation d’une interruption RX ; ce n’est pas l’activation du récepteur. |
| **UCSZ02** | Participe au format des données avec les bits de UCSR0C. |
| **RXB80 / TXB80** | Neuvième bit de données ; pas nécessaire pour les trames à 8 bits. |

## Scruter n’impose pas d’activer les interruptions

Une fonction peut lire l’indicateur de réception elle-même. Dans cette méthode, RXCIE0 et une routine d’interruption UART ne sont pas nécessaires.

Sources : datasheet, §19.7 p.152 ; §19.10.3 p.160 ; §13.3.3 p.70-71

# UCSR0A : prêt à recevoir ou à émettre ?

Une donnée reçue et une place libre pour émettre sont deux états différents.

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| RXC0 | TXC0 | UDRE0 | FE0 | DOR0 | UPE0 | U2X0 | MPCM0 |

| Indicateur | Ce que signifie 1 | Usage |
|---|---|---|
| **RXC0** | Au moins une donnée reçue reste à lire. | Côté réception |
| **UDRE0** | Le tampon d’émission est libre. | Côté émission |
| **TXC0** | L’émission de la trame est achevée et le tampon TX est vide. | Fin de transmission |

## RXC0 décrit le tampon, pas une impulsion




## Lire les données fait évoluer l’indicateur

RXC0 reste à 1 tant que des données non lues existent. Quand le tampon devient vide, le matériel le ramène à 0. RXC0 est en **lecture seule** : tu ne l’effaces pas avec une affectation ou un masque de remise à zéro.

**À retenir :** un tampon RX vide n’empêche pas le tampon TX d’être libre. Les deux directions sont indépendantes.

Sources : datasheet, §19.6.3 p.151 ; §19.7.3 p.154 ; §19.10.2 p.159-160

# UDR0 : même nom, deux sens

Lire accède au tampon RX ; écrire accède au tampon TX.

## Une lecture

Le CPU obtient un octet **reçu**.

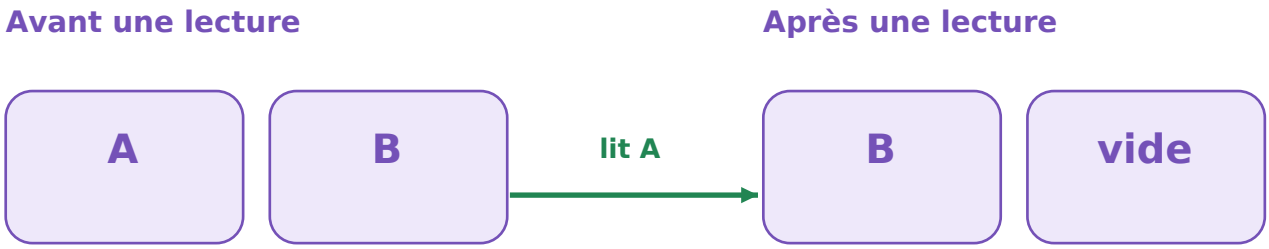
Cette lecture fait avancer la file du récepteur.

## Une écriture

Le CPU dépose un octet **à émettre**.

Cela n'injecte pas un caractère dans le tampon de réception.

## Réception : une petite file FIFO




**FIFO** : First In, First Out, premier arrivé, premier lu. Le tampon RX possède **deux niveaux**. Si B reste à lire après A, RXC0 reste à 1. Lire une seconde fois donne B, pas une copie de A.

## Un accès à UDR0 a un effet

N'utilise pas UDR0 pour « regarder sans consommer ». Une lecture retire une donnée de la file. Une lecture supplémentaire ou une lecture-modification-écriture peut donc perturber le parcours des données.

## Conserver le caractère reçu

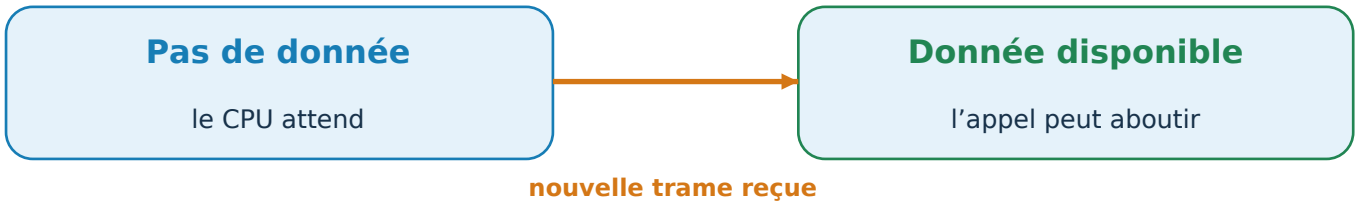
Une fois l'octet lu, son résultat doit rester disponible pour l'appelant et l'écho. Ne relis pas le registre à la place de la valeur déjà reçue : tu pourrais prendre le caractère suivant, ou lire alors qu'aucune donnée valide n'est annoncée.

Sources : datasheet, §19.2 p.143-144 ; §19.10.1 p.159

# Une fonction bloquante

Le programme reste dans l’appel tant qu’aucun octet n’est disponible.

## Deux moments dans un appel




| Outil de C | Sens général |
|---|---|
| `1 << k` | Construire un masque qui sélectionne la position k. |
| `R & masque` | Lire l’état des bits sélectionnés ; zéro s’ils sont tous à 0. |
| `!expression` | Vaut 1 si l’expression vaut 0, sinon vaut 0. |
| `while (condition)` | Répéter tant que la condition est vraie. |
| `return valeur;` | Terminer l’appel et donner une valeur à l’appelant. |

## Le matériel continue pendant l’attente

Le CPU vérifie un état, mais l’USART reconstitue les trames de façon autonome. Une attente par scrutation ne bloque pas le récepteur matériel. Elle occupe le CPU, ce qui est acceptable pour ce petit exercice. Les noms des registres viennent de **avr/io.h**.

## La fonction n’attend pas forcément à chaque appel

Si un octet est déjà dans le tampon, l’appel peut le fournir immédiatement. Aucun délai artificiel n’est nécessaire entre les caractères ; un délai trop long risque de remplir le tampon RX.

# Le même format dans les deux directions

On reprend ici 115200 8N1, dans la continuité des exercices précédents.

## 115200 : la cadence

115200 bits par seconde.

Le MCU et le terminal doivent utiliser des débits compatibles.

## 8N1 : la trame

8 bits de données.
Aucune parité.
1 bit de stop.

Start et stop sont gérés par l’USART.

**UBRR idéal = F_CPU / (K × débit demandé) - 1**

K = **16** en mode normal, **8** en double vitesse. Le registre est entier : compare les valeurs voisines, puis recalcule le débit réel **F_CPU / [K × (UBRR + 1)]**. F_CPU doit décrire la fréquence effective du MCU.

| Ce qu’il faut régler | Champ / emplacement |
|---|---|
| Mode asynchrone | UMSEL01:0 dans UCSR0C |
| Parité et stop | UPM01:0 et USBS0 dans UCSR0C |
| Taille des données | UCSZ02 dans B ; UCSZ01 et UCSZ00 dans C |
| Débit et facteur de division | UBRR0H/L ; U2X0 dans UCSR0A |

## En réception, la précision de l’horloge compte

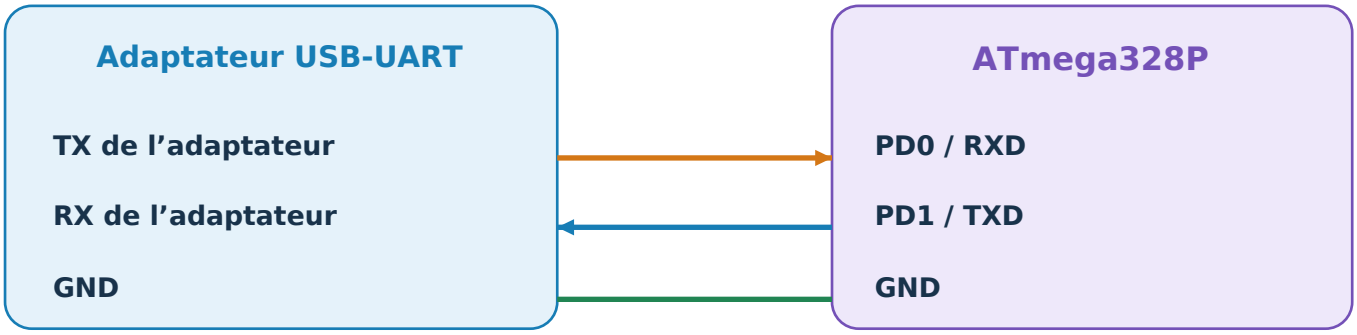
Le récepteur échantillonne à 16 fois le débit en mode normal et 8 fois en double vitesse. Compare l’erreur de débit et utilise une horloge suffisamment précise. Les limites de réception sont détaillées en p.155-157.

Sources : datasheet, tableau 19-1 p.146 ; §19.4 p.147-148 ; §19.10.4-19.10.5 p.161-162

# Deux lignes croisées et une masse

RX et TX sont nommés du point de vue de chaque appareil.

## Liaison logique avec un adaptateur USB-UART




Le premier fil transporte les caractères du PC vers le MCU. Le second transporte l’écho vers le PC. Le troisième fournit la référence électrique commune.

## Une carte avec USB-série intégré

Si l’interface est déjà sur ta carte, suis son schéma et utilise le connecteur prévu. Sinon, utilise un adaptateur UART logique avec des niveaux compatibles. Les signaux UART du MCU ne sont ni des signaux USB directs, ni ceux d’un port RS-232.

## RXD est une fonction alternative de PD0

L’USART prend le contrôle de cette broche quand le récepteur est activé. Les registres DDRD et PORTD ne remplacent pas la configuration de l’USART. Pour envoyer l’écho, la fonction TXD sur PD1 doit aussi être prête.

## Ce qui manquait au premier exercice TX seul

Pour voir un caractère envoyé par le MCU, une seule ligne de données pouvait suffire. Pour recevoir la frappe du PC **et** voir le retour, vérifie les deux lignes de données. Un écho absent peut venir du câblage dans un seul sens.

Sources : datasheet, §13.3.3 et tableau 13-9 p.70-71 ; schéma de liaison pédagogique

# Donnée reçue ≠ donnée forcément correcte

Ces indicateurs servent à comprendre et diagnostiquer, sans alourdir l’exercice de base.

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| RXC0 | TXC0 | UDRE0 | FE0 | DOR0 | UPE0 | U2X0 | MPCM0 |

| Indicateur | Ce qu’il signale |
|---|---|
| **FE0** | Erreur de trame : le premier bit de stop a été reçu à 0. Un mauvais débit peut en être une cause. |
| **DOR0** | Débordement de réception : des trames ont été perdues car le récepteur n’a pas été vidé assez vite. |
| **UPE0** | Erreur de parité, si la vérification de parité est activée. En mode sans parité, ce bit reste à 0. |

## État et donnée vont ensemble

Les indicateurs d’erreur sont associés à la donnée située en tête du tampon. Pour examiner les erreurs de cet octet, il faut lire **UCSR0A avant UDR0**. Lire UDR0 avance dans la file et change la donnée à laquelle l’état se rapporte.

## Le prototype ne décrit pas un retour d’erreur

**char uart_rx(void)** fournit un caractère. Il n’offre pas une seconde valeur distincte pour les erreurs. Pour l’exercice de base, conserve ce contrat ; n’ajoute pas arbitrairement un retour -1 qui pourrait représenter un octet valide.

## Éviter la perte de caractères

Ne désactive pas RX pour attendre : cela vide son tampon et peut perdre une trame en cours.

Sources : datasheet, §19.7.4-19.7.6 p.154 ; §19.10.2 p.159-160

# Du terminal aux observations

Afficher une touche localement ne prouve pas qu’elle a fait l’aller-retour.

## Ouvrir le bon port série

Exemple Linux : adapte le périphérique et garde les paramètres de l’UART.

```
screen /dev/ttyUSB0 115200,cs8,-parenb,-cstopb
```

Commence par des caractères simples, par exemple **A, b, 7**, un à un. Observe le retour. Une touche Entrée ou un caractère accentué ne se résume pas forcément à un caractère visible transmis.

| Observation | Interprétation à vérifier |
|---|---|
| Rien n’apparaît | Port ; fils ; masse ; RXEN0 et TXEN0 ; débit ; indicateur testé. |
| Deux copies d’une touche | Écho local du terminal ajouté au retour du MCU, ou double émission dans le programme. |
| Caractères illisibles | Format ou débit incompatibles ; F_CPU incorrect ; niveaux électriques. |
| Caractères perdus en rafale | Tampon RX trop peu lu ; pauses longues ; DOR0 ; autres tâches lentes. |
| Entrée ou retour arrière surprenant | Caractères de contrôle renvoyés tels quels ; comportement du terminal. |

## Un écho d’octets ne transforme pas le texte

En UTF-8, une lettre accentuée peut utiliser plusieurs octets. Un appel à uart_rx traite un octet, pas forcément un symbole affiché. Ne rajoute pas automatiquement de saut de ligne et ne remplace pas les contrôles si l’objectif est de renvoyer ce qui a été reçu.

# Construire ta propre solution

Complète les décisions et les questions avant d’assembler ton programme.

| Décision | Ta réponse / tes repères |
|---|---|
| Broche de réception / d’émission | _____ |
| Bit qui active le récepteur | _____ |
| Bit qui active l’émetteur | _____ |
| Indicateur d’une donnée à lire | _____ |
| Indicateur d’une place libre en TX | _____ |
| Registre de données / effet d’une lecture | _____ |
| Condition générale de l’attente | _____ |
| Valeur que la fonction doit retourner | _____ |
| Comment conserver la valeur reçue | _____ |
| Comment la fournir à uart_tx | _____ |
| F_CPU ; UBRR ; format partagé | _____ |

## Quatre questions pour vérifier que tu comprends

1. Pourquoi tester UDRE0 ne dit-il pas si le PC a envoyé un caractère ?
2. Pourquoi lire deux fois UDR0 peut-il donner deux caractères différents ?
3. Si un octet de valeur 0 arrive, faut-il quand même le renvoyer ?
4. Si RXC0 reste à 1 après une lecture, est-ce forcément une erreur ?

**Critères de réussite :** réception bloquante d’un octet, valeur correctement retournée, caractère reçu renvoyé sans transformation, répétition du traitement.

Sources à relire : p.150, p.152, p.154, p.159-162 de la datasheet fournie

# Où lire dans ta datasheet

ATmega328P • Document 7810D-AVR-01/15 • Les pages PDF et imprimées coïncident.

| Ce que tu cherches | Section / tableau | Page(s) |
|---|---|---|
| Broches RXD et TXD ; priorité GPIO | §13.3.3 ; tableau 13-9 | 70-71 |
| Deux directions indépendantes | §19.2 ; figure 19-1 | 143-144 |
| Formules de débit ; double vitesse | §19.3.1-19.3.2 ; tab.19-1 | 145-146 |
| Start, données, stop et format | §19.4 ; figure 19-4 | 147-148 |
| Initialisation commune | §19.5 | 148 |
| Émission avec uart_tx | §19.6.1 ; §19.6.3 | 150-151 |
| Activation RX ; réception de 5 à 8 bits | §19.7 ; §19.7.1 | 152 |
| Sens de RXC et lecture du tampon | §19.7.3 | 154 |
| Erreurs ; arrêt du récepteur | §19.7.4-19.7.6 | 154 |
| Échantillonnage ; tolérance de débit | §19.8.1-19.8.3 | 155-157 |
| UDR : RX en lecture, TX en écriture | §19.10.1 | 159 |
| Indicateurs A ; activations B | §19.10.2-19.10.3 | 159-160 |
| Format C et partage UBRR haut/bas | §19.10.4-19.10.5 | 161-162 |

## Comment lire la source sans recopier la correction

La page **152** contient les explications et un exemple de fonction de réception. Ce guide décrit les mécanismes sans reproduire ce code. Les noms génériques de la datasheet utilisent **n** : pour USART0, UCSRnA devient UCSR0A et UDRn devient UDR0.

Les rappels sur les prototypes et return relèvent du langage C. Pour les options du terminal : Ouvrir le manuel officiel GNU Screen : Window Types

**Parcours conseillé :** réception (p.1-7) → configuration et câblage (p.8-9) → erreurs et tests (p.10-11) → feuille à compléter (p.12).